

Estructura de Datos
Act-1.3.2 Notación Asintótica (Funciones Recursivas)

Nombre: Matricula: **Elena María Barrios JordánA01771338**

1) ¿Cuál es el orden de cada uno de los siguientes algoritmos?

a) //Entrada: Un entero positivo (n)

```
int algo(int n) {  
    if (n <= 0)  
        return 400;  
    else  
        return algo(n-2) + algo(n-2) + algo(n-2) + algo(n-2); }  
}
```

$O(2^n)$

b) //Entrada: Un entero positivo (n)

```
int algo(int n) {  
    if (n <= 0)  
        return 123;  
    else  
        return algo(n-4) + algo(n-4) + algo(n-4);  
}
```

$O(3^{(n/4)})$

c) //Entrada: Un entero positivo (n)

```
int algo(int n) {  
    if (n == 0)  
        return 400;  
    else  
        return algo(n/2) + algo(n/2) + algo(n/2) + algo(n/2); }  
}
```

$O(n^{(\log(\text{base}))2^4})$

d) //Entrada: Un entero positivo (n)

```

int algo(int n) {
    if (n == 0)
        return 123;
    else
        return algo(n/4) + algo(n/4) + algo(n/4);
}

```

$O(n^{(\log(\text{base}))4^3})$

2) ¿Cuál es la fórmula recursiva del tiempo de ejecución de cada uno de los siguientes algoritmos?

a) //Entrada: Un entero positivo (n)

```

int algo(int n) {
    if (n <= 0)
        return 400;
    else
        return algo(n-2) + algo(n-2) + algo(n-2) + algo(n-2); }

```

$T(n) = 4T(n-2) + 1$, porque si n es menor a 0 solo le regresa un valor con un solo resultado pero si no, llama 4 veces al $\text{algo}(n-2)$ y hace 3 sumas. Y sabemos que hay tiempo constante por la condición base.

b) //Entrada: Un entero positivo (n)

```

int algo(int n) {
    if (n <= 0)
        return 123;
    else
        return algo(n-4) + algo(n-4) + algo(n-4);
}

```

$T(n) = 3T(n-4) + 1$, este caso es muy similar al anterior y también hay tiempo constante por $n \leq 0$.

c) //Entrada: Un entero positivo (n)

```

int algo(int n) {
    if (n == 1)
        return 400;
    else
        return algo(n/2) + algo(n/2) + algo(n/2) + algo(n/2); }

```

$T(n) = 4T(n/2) + 1$, aquí es un poco diferente porque la comparación la hace con el número

1 en lugar de hacerla con el 0, es como $T(1)=1$.

d) //Entrada: Un entero positivo (n)

```
int algo(int n) {  
    if (n == 1)  
        return 123;  
    else  
        return algo(n/4) + algo(n/4) + algo(n/4);  
}
```

$T(n) = 3T(n/4) + 1$ igual es una condición que solo hay una comparación para la que solo hay un return.